



Programación - Guía de Ejercicios N°3

1. El Tour de Viña del Mar, en el cual participan los mejores ciclistas de la zona, siempre ha tenido problemas con el registro de los tiempos (en formato horas:minutos), ya que estos son anotados por etapa y luego necesitan cronometrar el tiempo total. El encargado de logística del certamen ha decidido recurrir a los ingeniosos alumnos de Programación de la USM para que le solucione el problema. Para esto el requiere de las siguientes funciones:

- a) La función `tiempo_por_ciclista(t)` que calcule el tiempo total del ciclista de sus tiempos cronometrados por etapa `t`.

```
ta = '1:20 2:10 1:24 0:54 3:36'
print tiempo_por_ciclista(ta)
9:24
```

- b) La función `almacenar_tiempos(m)` que retorne un diccionario, donde la llave es el apellido del ciclista, y el valor el tiempo cronometrado de los `m` participantes.

```
print almacenar_tiempos(3)
Participante 1: Rayo Caracol
Tiempos: 2:30 1:45 1:50 2:23 1:54

Participante 2: Speedy Perez
Tiempos: 1:30 2:25 2:30 1:13 1:12

Participante 3: Flash Flacon
Tiempos: 1:40 2:15 1:30 1:43 0:43

{'Caracol':10:22, 'Perez':8:50, 'Flacon':7:51}
```

2. Una prestigiosa universidad mantiene una lista de alumnos en un archivo `alumnos.txt`, donde se almacenan el RUT y el nombre de los alumnos en el formato `rut:nombre`. Además, por cada alumno, hay un archivo `rut.txt`, donde el nombre es el RUT del alumno. Este último tiene los ramos tomados por el alumno y las notas obtenidas en el formato `ramo,nota1 nota2 ...`, como muestra el siguiente ejemplo:

alumnos.txt	
150870162:	Juan Duran
167754321:	Lorena Perez
156432876:	Alvaro Ruiz
167844261:	Marcelo Pinto

150870162.txt	
INS-131,	5.6 6.5 6.6
MAT-100,	5.8 1.4 1.5 4.1
FIS-110,	1.0 6.3
INS-141,	4.9 5.6 6.3 4.3 4.9

En el ejemplo, además existen los archivos `167754321.txt`, `156432876.txt` y `167844261.txt` con sus respectivos ramos y notas.

- a) Desarrolle la función `notas(archivo,nombre)` que recibe el archivo de alumnos y un nombre en particular. La función debe retornar un diccionario donde las llaves son las siglas de las asignaturas y los valores son listas con las notas obtenidas por el alumno.

```
print notas('alumnos.txt', 'Juan Duran')
{'INS-141': [4.9, 5.6, 6.3, 4.3, 4.9], 'INS-131': [5.6, 6.5, 6.6],
'MAT-100': [5.8, 1.4, 1.5, 4.1], 'FIS-110': [1.0, 6.3]}
```

- b) Desarrolle la función `mejor_alumno(archivo,ramo)` que recibe el archivo de alumnos y la sigla de un ramo en particular. Esta función debe retornar el nombre del alumno con el mejor promedio obtenido en tal ramo. Si el ramo no ha sido tomado por algún alumno, considerar nota 0 en este caso.

```
print mejor_alumno('alumnos.txt', 'INS-131')
Juan Duran
```

- c) Desarrolle la función `agregar_ramo(rut,ramo,notas)` que recibe el RUT del alumno, la sigla de un ramo y una lista con las notas de ese ramo. Esta función debe agregar dicho ramo y sus notas al archivo del alumno.

```
agregar_ramo('150870162', 'DEP-010', [7.0, 4.5, 5.2])
```

150870162.txt

```
INS-131,5.6 6.5 6.6
MAT-100,5.8 1.4 1.5 4.1
FIS-110,1.0 6.3
INS-141,4.9 5.6 6.3 4.3 4.9
DEP-010,7.0 4.5 5.2
```

3. Andronico Bank es un banco muy humilde que hasta hace poco usaban solo papel y lápiz para manejar toda la información de sus clientes, también humildes. Como una manera de mejorar sus procesos, Andronico Bank quiere utilizar un sistema computacional basado en Python. Por eso se traspassa la información de sus clientes a un archivo de texto, indicando su rut, nombre y clase cliente. El archivo `clientes.txt` es un ejemplo de lo anterior.

clientes.txt

```
9234539-9;Sebastian Davalos;VIP
11231709-k;Choclo Delano;Pendiente
5555555-6;Sebastian Pinera;VIP
9999999-k;Gladis Maryn;RIP
12312312-1;Michel Bachelet;VIP
8888888-8;Companero Yuri;Estandar
7987655-1;Sergio Estandarte;RIP
```

A usted se le pide lo siguiente:

- a) Implemente la función `buscar_clientes(archivo,clase)` que reciba como parámetros el nombre del archivo de clientes y una clase, y retorne un diccionario con los rut de los clientes como llaves y los nombres como valor de todos los clientes pertenecientes a la clase entregada como parámetro.

```
print buscar_clientes('clientes.txt', 'Pendiente')
{'11231709-k': 'Choclo Delano'}
```

- b) Implemente la función `dar_credito(archivo, rut)` que reciba como parámetros el nombre del archivo de clientes y el rut de un cliente, y que retorne `True` si este es VIP o `False` si no lo es. Si no encuentra el cliente la función retorna `False`.

```
print dar_credito('clientes.txt', '9999999-k')
False
```

- c) Implemente la función `contar_clientes(archivo)` que reciba como parámetros el nombre del archivo de clientes y que retorne un diccionario con la cantidad de clientes de cada clase en el archivo.

```
print contar_clientes('clientes.txt')
{'VIP': 3, 'Pendiente': 1, 'RIP': 2, 'Estandar': 1}
```

4. La importadora automotriz El PALETA S.A. mantiene en un archivo de texto la información de los autos que tiene a la venta en la actualidad en el archivo `todos.txt`, esta consiste en un código de identificación, la marca, el modelo, el año, el color y su precio. A continuación se muestran algunos de los registros de este archivo.

`todos.txt`

```
1001;kia;sorento;2014;azul;10 millones
1002;kia;sportage;2015;gris;8 millones
1003;ford;explorer;2016;rojo;11 millones
1004;ford;ecosport;2013;azul;6 millones
1005;hyundai;elantra;2014;verde;7 millones
1006;hyundai;tucson;2012;blanco;9 millones
1007;hyundai;accent;2015;negro;6 millones
```

- a) Implemente la función `nuevos_autos(archivo)` que recibe el nombre de un `archivo`, con las mismas características que `todos.txt`, el cual contiene los autos nuevos que han llegado a la importadora. Esta función debe actualizar el archivo `todos.txt` agregando los nuevos autos.
- b) Implemente la función `seguro(archivo)`, que recibe un `archivo` de las características de `todos.txt` y que genere un archivo denominado `pagos.txt`, donde en cada línea debe ir el código, la marca y el valor de la prima de seguro a pagar del auto. La prima es de un 1 % para los autos del año y de un 0.8 % para autos de otros años.
- c) La importadora tiene un archivo con la información de sus clientes: rut, nombre, correo electrónico y la marca de su preferencia. A continuación se muestran algunos de los registros del archivo `clientes.txt`. Implemente la función `preferencia(archivo_autos, archivo_clientes, rut)` que reciba el nombre de los archivos que contienen la información de los autos y de los clientes y el rut de un cliente en particular y que retorne una lista con todo los modelos de su marca preferida.

`clientes.txt`

```
12467341-2:Juan Perez:perezj@gmail.com:hyundai
10225467-5:Luis Solis:luchosol@hotmail.com:kia
13458832-1:Mary Lu:marilu@hotmail.com:ford
```

5. Desde 2007, geográficamente Chile está dividido en 15 regiones desde Arica hasta la Antártica Chilena. Además de la división geográfica, también existen las regiones naturales (zonas), las cuales son cinco:

- Norte Grande
- Norte Chico
- Zona Central
- Zona Sur
- Zona Austral

La tarea que se le encomienda a Ud., es construir ciertas funciones que ayuden a obtener información con respecto a estas divisiones, para esto, Ud. cuenta con la información de cada región en el archivo `regiones.txt`, donde en cada línea se encuentra el número de la región, el nombre, la cantidad de habitantes, la superficie (en km²) y la capital; es importante saber que las regiones están ordenadas de manera ascendente según su número. Por otro lado, Ud. cuenta con la información de cada región natural en el archivo `zonas.txt`, en el cual cada línea se encuentra la zona natural junto con las regiones que pertenecen a ella.

`regiones.txt`

```
I,Tarapaca,300021,42225,Iquique
II,Antofagasta,547463,126049,Antofagasta
...
XIV,Los Rios,364592,18429,Valdivia
XV,Arica y Parinacota,213816,16873,Arica
```

`zonas.txt`

```
norte grande:XV,I,II
norte chico:III,IV
centro:V,XIII,VI,VII,VIII
sur:IX,XIV,X
austral:XI,XII
```

- a) Desarrolle la función `regiones_naturales(arch)` que recibe el nombre con la información de las zonas `arch` y retorne un diccionario, donde la llave sea la zona y el valor sea una lista con las regiones que la forman.

```
print regiones_naturales('zonas.txt')
{'austral': ['XI', 'XII'], 'sur': ['IX', 'XIV', 'X'], 'norte grande':
['XV', 'I', 'II'], 'centro': ['V', 'XIII', 'VI', 'VII', 'VIII'],
'norte chico': ['III', 'IV']}
```

- b) Desarrolle la función `habitantes_por_zona(arch1,arch2)` que recibe el nombre del archivo con la información de las regiones `arch1` y el nombre del archivo con la información de las zonas `arch2` y retorne un diccionario, donde la llave sea la zona y el valor sea la cantidad de habitantes para tal zona.

```
print habitantes_por_zona('regiones.txt','zonas.txt')
{'norte chico': 999708, 'centro': 12238720, 'norte grande': 1061300,
'sur': 2075798, 'austral': 259077}
```

- c) Después de la tercera guerra mundial, el país ha sufrido modificaciones en su geografía, por supuesto a su favor, por lo que es necesario actualizar tanto el archivo `regiones.txt` como `zonas.txt`, para esto desarrolle la función

`agregar_reg(arch1,arch2,nombre,sup,hab,capital,zona)`, la cual debe agregar a ambos archivos

(`arch1,arch2`) la nueva región que ha ganado el país con la información entregada: nombre de la región, superficie (en km^2), habitantes, capital y a la región natural que va a pertenecer. **Nota:** cada vez que se agrega una región, le corresponde el número que sigue; por ejemplo la primera región en agregar será la decimosexta región. Asuma que existe la función `entero_a_romano(x)` que convierte el entero `x` a número romano.

```
agregar_reg('regiones.txt','zonas.txt','Mendoza',148827,1863809,
'Mendoza','centro')
```

`regiones.txt`

```
I,Tarapaca,300021,42225,Iquique
...
XV,Arica y Parinacota,213816,16873,Arica
XVI,Mendoza,148827,1863809,Mendoza
```

`zonas.txt`

```
norte_grande:XV,I,II
norte_chico:III,IV
centro:V,XIII,VI,VII,VIII,XVI
sur:IX,XIV,X
austral:XI,XII
```

6. La automotora PyCar requiere generar información importante con respecto a sus ventas. Principalmente requiere realizar un análisis sobre el desempeño de sus vendedores así como también sobre la evolución de las ventas que se van realizando.

Para lograr este propósito, PyCar tiene almacenado en el archivo `productos.txt`, la información del precio de los automóviles que tiene a la venta. Cada producto viene especificado por la marca del automóvil, el modelo y el precio de venta. Por otro lado, PyCar almacena en el archivo `ventas.txt`, cada una de las transacciones realizadas por concepto de la venta de un automóvil, esta información cuenta con el día de la venta, la hora en que se ha realizado, el modelo del automóvil, la marca del mismo y el código del vendedor que ha realizado la transacción.

Nota: los archivos son a modo de ejemplo, su programa debe funcionar para toda la cantidad de automóviles que puede tener la automotora y para todas las ventas que se han realizado durante el año 2014.

`productos.txt`

```
hyundai,accent,7390000
hyundai,tucson,11690000
hyundai,santafe,15790000
suzuki,nomade,9990000
suzuki,vitara,9490000
suzuki,swift,7190000
toyota,yaris,7890000
toyota,corolla,9390000
toyota,rav4,12090000
```

`ventas.txt`

```
2014/9/30,19:45,suzuki,vitara,5678
2014/9/30,20:15,toyota,corolla,4523
2014/9/30,10:40,suzuki,nomade,6547
2014/9/30,14:40,suzuki,vitara,5678
2014/10/1,10:40,toyota,yaris,4523
2014/10/1,14:40,suzuki,vitara,6547
2014/10/2,10:40,hyundai,accent,6547
2014/10/2,12:40,suzuki,vitara,5678
2014/10/2,13:20,hyundai,santafe,4523
```

`venta_diaria.txt`

```
2014/9/30:38360000
2014/10/1:17380000
2014/10/2:32670000
```

Con el fin de ayudar a PyCar en sus requerimientos, Ud. debe realizar lo siguiente:

- a) Desarrolle la función `precios(archivo)` que recibe el nombre del archivo con la información de los productos `archivo` y retorne un diccionario, donde la llave sea el modelo del automóvil y el valor sea el precio del mismo.

```
print precios('productos')
{'rav4': 12090000, 'corolla': 9390000, 'accent': 7390000,
 'nomade': 9990000, 'yaris': 7890000, 'tucson': 11690000,
 'santafe': 15790000, 'swift': 7190000, 'vitara': 9490000}
```

- b) Desarrolle la función `total_ventas_mes(codigo,mes,archivo1,archivo2)` que recibe el código de un vendedor, un mes en particular, el nombre del archivo con la información de los productos `archivo1` y el nombre del archivo con la información de las ventas `archivo2` y retorne el total de ventas que ha realizado el vendedor en ese mes.

```
print total_ventas_mes(4523,10,'productos','ventas')
23680000
```

- c) Desarrolle la función `venta_diaria(archivo1,archivo2)` que recibe el nombre del archivo con la información de los productos `archivo1` y el nombre del archivo con la información de las ventas `archivo2` y genere el archivo `venta_diaria.txt` indicando la venta total que se ha realizado por día.